

# Math-heuristics for Stamps

Now you have 400 stamps and 2000 bids, given in the data-file `bid_data_400_2000.jl`

## Problem

- Maximize the profit obtained by selling your stamps, using the Hamming-Distance-Heuristic.

The model is identical with the Stamps model. Notice that we do not need a slack variable, the solution with zero accepted bids is feasible, but obviously not optimal.

## Problem 1

- Implement the Hamming-Distance-Heuristic algorithm for the Stamps problem. You can start with the version used in the Mathematical Heuristics for Wedding Planning problem.

The full model for Hamming-Distance-Heuristic algorithm for Stamps in Julia/JuMP is given below:

```
start_time=time_ns()
#####
# Stamps Assignment, "Mathematical Programming Modelling" (42112)
# utilizing Hamming-distance heuristic optimization
#####
# Intro definitions
using JuMP
using HiGHS
#####

let
#####
# Data
include("bid_data_400_2000.jl")
```

```

(B,S)=size(BidSets)
println("Bids: $B   stamps: $S")
time_limit=120
#####

#####
# Model
stamps = Model(HiGHS.Optimizer)
set_silent(stamps)
# accept bid or not, yes if x[b]=1
@variable(stamps, x[1:B], Bin)

# Max sales profit
@objective(stamps, Max, sum( BidPrice[b] * x[b] for b=1:B) )

@constraint(stamps, [s=1:S],
            sum( BidSets[b,s]*x[b] for b=1:B) <= 1 )
#####

#####

function AddKConstraintAndOptimize(xVal,K)

    @constraint(stamps, Kconstraint,
                sum( x[b] for b=1:B if xVal[b]==0) +
                sum( (1-x[b]) for b=1:B if xVal[b]==1)
                <= K)
    optimize!(stamps)
    xVal=round.(Int,value.(x))
    curObj=round(Int64,objective_value(stamps))
    delete(stamps, Kconstraint)
    unregister(stamps, :Kconstraint)
    return (curObj,xVal)
end

println("Starting Hamming Distance Heuristic")
xVal=zeros(Int8,B)
K=1
it=1
curObj=-1
oldObj=-1

while (time_ns()-start_time)/1.0e9<time_limit
    (curObj,xVal)=AddKConstraintAndOptimize(xVal,K)
    this_time=(time_ns()-start_time)/1.0e9
    println("It: $(it) Time: $(round(this_time,digits=1)) K: $(K) CurObj: $(curObj)")

```

```

        if curObj<=oldObj # comparison for maximization problems
            K=K+1
        else
            K=1
        end
        oldObj=curObj
        it+=1
    end
    *****

    *****
println("Successfull end of $(PROGRAM_FILE)")
    *****
end

```

## Problem 2

- Implement the Fix-And-Optimize algorithm for the Stamps problem. You can start with the version used in the Mathematical Heuristics for Wedding Planning problem.

The full model for Hamming-Distance-Heuristic algorithm for Stamps in Julia/JuMP is given below:

```

start_time=time_ns()
*****
# WeddingPlanner Assignment, "Mathematical Programming Modelling" (42112)
# optimized using FixAndOptimize
*****
# Intro definitions
using JuMP
using HiGHS
using Random
*****

let
    *****
    # Data
    include("bid_data_400_2000.jl")
    (B,S)=size(BidSets)
    println("Bids: $B   stamps: $S")
    time_limit=120
    K=100
    *****

```

```

*****
# Model
stamps = Model(HiGHS.Optimizer)
set_silent(stamps)
# accept bid or not, yes if x[b]=1
@variable(stamps, x[1:B], Bin)

# Max sales profit
@objective(stamps, Max, sum( BidPrice[b] * x[b] for b=1:B) )

@constraint(stamps, [s=1:S],
            sum( BidSets[b,s]*x[b] for b=1:B) <= 1 )
*****

*****

function FixAndOptimize(xVal,K)
    # first lock all values
    for b=1:B
        fix(x[b], xVal[b]; force = true)
    end

    # then free K
    free_vars=0
    while free_vars<K
        b=rand(1:B)
        if is_fixed(x[b])
            unfix(x[b])
            free_vars+=1
        end
    end

    optimize!(stamps)
    xVal=round.(Int,value.(x))
    cur_obj=round(Int64,objective_value(stamps))
    return (cur_obj,xVal)
end

println("Starting heuristic")
cur_xVal=zeros(Int8,B)

# set the fix and optimize neighborhood
#K=round(Int64,G*T/2)

it=1
cur_obj=-1

```

```

old_obj=-10000000000000000
this_time=(time_ns()-start_time)/1.0e9

while this_time<time_limit
    (cur_obj,cur_xVal)=FixAndOptimize(cur_xVal,K)
    this_time=(time_ns()-start_time)/1.0e9
    println("It: $(it) Time: $(round(this_time,digits=1)) K: $(K) CurObj: $(cur_obj) OldObj: $(old_obj)")
    if cur_obj-old_obj < 0
        println("This should not happen")
        exit(0)
    end

    if cur_obj>old_obj+0.5 # comparison for maximization problems
        old_obj=cur_obj
    end

    it+=1
end
#####

#####
println("Successfull end of $(PROGRAM_FILE)")
#####
end

```